# CMPE311 Project 1: Asynchronous Tasking Lab Report

**Document name:** CMPE311 Project 1: Asynchronous Tasking Lab Report

**Document reference:** DulceCMPE311.project1.AsynchTasking

**Date of publication:** 9/21/2026

**LEAD ENGINEER:** Dulcey Comeau, UMBC

## STAKEHOLDERS

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

**HIGH-LEVEL DESCRIPTION:** An Arduino Uno R3-based embedded system that blinks two LEDs at independently User-configurable intervals set over the serial-monitor, with serial I/O handled asynchronously so it never disturbs the blinking.

**DESCRIPTION:** This project implements an embedded system on an Arduino Uno R3-compatible board (ELEGOO Uno R3, ATmega328P) that drives two LEDs, one on digital pin 2 and one on digital pin 3. Through the Arduino IDE serial-monitor, The User selects a LED (1 or 2) and enters a blink interval in milliseconds; the selected LED then blinks with an on-time and off-time each equal to half the entered interval. Serial prompting and reading are implemented as a non-blocking state machine, and each LED is driven by an independent millis()-based scheduler, so waiting for User input never pauses or perturbs either LED, and changing one LED never affects the other. All timekeeping and concurrency use bare-metal, non-blocking techniques from the Arduino core; no real-time operating system or third-party multitasking library is used.

**RESULT SUMMARY:** The delivered design satisfies every high-level technical requirement (HL.1-HL.6 of Table 3) and educational constraint (EC.1-EC.5 of Table 4) by construction, and is exercised against the testing requirements (T.0-T.10) derived in Part II. The two LEDs blink independently at the User-set intervals, serial I/O is confirmed asynchronous (blinking continues while a prompt awaits input), and setting one LED leaves the other unaffected, reproducing the Table 5 dialog. The software compiles with the gcc toolchain shipped in Arduino IDE 2.3.3 for the ATmega328P and runs unmodified on the Reference-platform. Measured timing data, photographs, and video evidence for each test are recorded in the Testing Results section.

# REFERENCES AND GLOSSARY

## REFERENCES

- How to do Multitasking with the Arduino (non-blocking / “Blink Without Delay” pattern).
- The Arduino Handbook - Learn Microcontrollers for Embedded Systems.
- sec04b-engineeringProcess_pt1.pdf - Lecture on the Engineering Process, Part 1 of 2.
- sec04b-engineeringProcess_pt2.pdf - Lecture on the Engineering Process, Part 2 of 2.
- CMPE310 asynchronous Arduino lab handout (LAB: PROJECT-ASYNC).

Project source repository: *[insert your GitHub repository URL here]* (per EC.2).

## DEFINITIONS

**“The User” -** The person operating (not programming) the embedded system.

**“The System” -** The embedded system being operated by The User.

**“The Customer” -** The person(s) paying for the embedded system being designed and built.

**“The Developer” -** The person(s) designing and building the System.

**“The Evaluator” -** The person(s) that determine whether or not The System satisfies The Customer-requirements.

**“The Customer-requirements” -** The requirements defined by The Customer as satisfying The Contract.

**“The HL-requirements” -** The System’s high-level technical requirements derived from The Customer-requirements.

**“The Requirements” -** Context dependent; e.g. may refer to the HL-requirements or Customer-requirements.

**“The Educational-constraints” -** The requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.

**“The Company” -** The organization The Customer has contracted with to build The System.

**“The Contract” -** The business document that legally binds The Company to provide some service or product to The Customer.

**“serial-monitor” -** The serial port used by the Arduino IDE to communicate with The User.

**“The Reference-platform” -** The configuration of The System used by The Developer to test and validate The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.

## ACRONYMS AND ABBREVIATIONS

**Arduino -** an Italian open-source hardware and software company; also refers to a development board created by that company.

**arduino.h -** header for a library of convenience functions specific to the Arduino development platform.

**AVR -** A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology.

**avr.h -** header including helper functions and definitions for using an AVR microcontroller.

**ELEGOO -** A Chinese company that develops and markets 3D printers and accessories.

**HL -** High-level.

**EC -** Educational Constraint.

**IDE -** Integrated Development Environment.

**gcc -** front end for the GNU Compiler Collection.

**Github -** A widely used distributed SVC (Software Version Control) system.

**LED -** Light Emitting Diode.

# REQUIREMENTS

## CONVENTIONS

**Must, shall or will -** the design must satisfy the requirement.

**May -** the design may satisfy the requirement but doesn't have to.

**Inform / Informative -** the description is provided for understanding only and contains no requirement.

Customer requirements are labeled "C#". Provided high-level technical requirements are labeled "HL.#[.#]". Educational constraints are labeled "EC.#[.#]". Testing/validation requirements are labeled "T.#[.#]". The high-level requirements derived in Part I are labeled "HL-I.#" to distinguish them from the provided Part II requirements that the delivered System implements.

## Part I: Customer Requirements and Derived High-Level Requirements

**Note:** *Per the assignment, the high-level requirements derived here (HL-I.#) are the Part I deliverable and are **not** used in Parts II or III. The built System (Parts II-III) implements the separate provided requirements in Table 3.*

**Table 1. Customer Requirements (given)**

| ID | Customer requirement |
|---|---|
| **C1** | The User will be able to choose the blink rate of the LED with a button. |
| **C2** | The User will choose between 4 rates plus off. |
| **C3** | The blink rates must be set by an operator prior to the system being started. |
| **C4** | Permitted blink rates must be perceivable by the viewer. |
| **C5** | The minimum blink rate must be no slower than once every 5 seconds. |
| **C6** | The System must run upon an Arduino Uno R3 compatible board. |

**Derived High-Level Technical Requirements (Part I)**

**HL-I.1** The System must run on an Arduino Uno R3-compatible board based on the ATmega328P microcontroller.

**HL-I.2** The System must use a single momentary push-button as the sole User control for selecting the active blink state.

**HL-I.2.1** A button press must be debounced so that one physical press advances the selection by exactly one state.

**HL-I.3** The System must provide exactly five selectable states: four distinct non-zero blink rates and one "off" state in which the LED is continuously de-energized.

**HL-I.3.1** Successive button presses must cycle through the five states in a fixed, repeating order.

**HL-I.4** The four blink rates must be fixed values established before the System is started (e.g. compile-time constants) and must not be changeable by The User during operation.

**HL-I.5** Every non-zero blink rate must be perceivable as distinct on/off transitions; therefore the fastest blink rate must be no faster than 4 Hz (period ≥ 250 ms, each on/off phase ≥ 125 ms).

**HL-I.6** The slowest non-zero blink rate must have a period no greater than 5 s (frequency ≥ 0.2 Hz).

**HL-I.6.1** Consequently, every configured blink period must lie within the closed interval [250 ms, 5000 ms].

## Part II: Provided High-Level Requirements and Educational Constraints

The following requirements are provided by the instructor and are the requirements the delivered System (Parts II-III) must satisfy.

**Table 3. High-level technical requirements to be used**

| ID | High-level technical requirement |
|---|---|
| **HL.1** | The System must use at least 2 LEDs. |
| **HL.2** | The System must use a standard Arduino compatible development board (e.g. the provided ELEGOO Uno R3). |
| **HL.3** | The System must communicate with The User via the Arduino IDE serial-monitor port. |
| **HL.4** | Any use of the serial-monitor in HL.3 must be asynchronous and not affect the blinking of the LEDs. |
| **HL.5** | The User must set the blink interval of the LEDs in msec. |
| **HL.6** | The blink rate of each LED must be constant unless changed by The User. |

**Table 4. Educational Constraints**

| ID | Educational constraint |
|---|---|
| **EC.1** | The Developer must use the version of gcc used by Arduino IDE 2.3.3 or later for the AVR ATmega328p to cross compile any program used by The System. |
| **EC.2** | The Developer must have any software used in The System hosted on a github repository. This software does not need to include libraries, macros or other code not developed by The Developer. |
| **EC.2.1** | The Evaluator must be able to (a) download from the repository any software, documentation and other material needed for their evaluation, (b) cross compile any software, and (c) run that software on The Reference-platform. |
| **EC.3** | LEDs must be connected to Arduino digital Pins 2 & 3. |
| **EC.4** | The Evaluator must be able to successfully have The System perform the dialog in Table 5. |
| **EC.5** | The Developer must do bare metal programming, i.e. they cannot use any 3rd party multi-tasking libraries or programs such as FreeRTOS. |

**Table 5. Example dialog (User inputs shown in the right column)**

| Serial port I/O | User input / notes |
|---|---|
| What LED? (1 or 2) | 2 |
| What interval (in msec)? | 600: LED2 starts blinking at 300 ms on / 300 ms off. LED1 is unaffected. |
| What LED? (1 or 2) | 1 |
| What interval (in msec)? | 1600: LED1 starts blinking at 800 ms on / 800 ms off. LED2 is unaffected. |

| **Serial port I/O** | **User input / notes** |
| --- | --- |
| What LED? (1 or 2) | (waiting for next LED# / interval pair) |

# DESIGN

## DESIGN PRE-REQUISITES

1. Reference-platform: an Arduino Uno R3-compatible board (ELEGOO Uno R3, ATmega328P at 16 MHz) with a USB 2.0 type-B to type-A cable to a host computer.
2. Two LEDs, each in series with a 470 Ω current-limiting resistor, connected to digital pins 2 and 3 (EC.3).
3. Arduino IDE 2.3.3 or later with the Arduino AVR Boards core installed, which supplies the gcc cross-compiler for the ATmega328P (EC.1).

## DEVELOPMENT PLATFORM

1. Arduino IDE 2.3.3 or later (Arduino AVR Boards package; target board: Arduino Uno / ATmega328P).
2. Host computer (Windows, macOS, or Linux) with a USB port.
3. Source code is maintained in a public GitHub repository (EC.2) so the Evaluator can download, cross-compile, and run it (EC.2.1).
4. Serial-monitor set to 9600 baud with the line ending set to “Newline” (or “Both NL & CR”).

## CIRCUIT SCHEMATIC

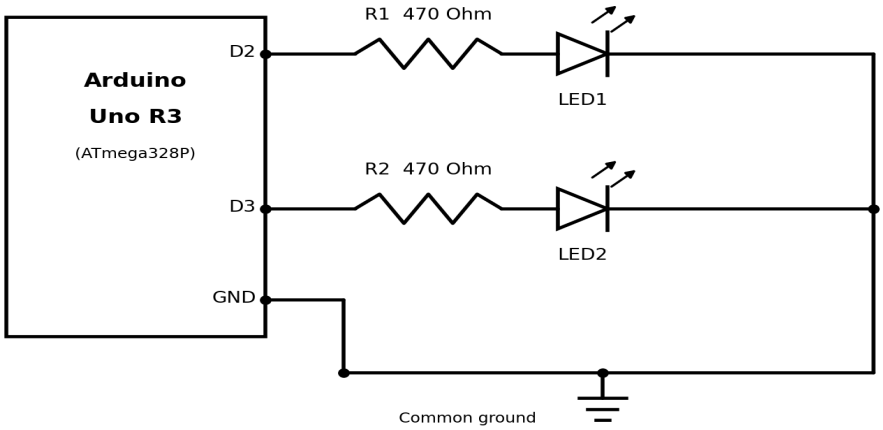



**Figure 1. Testbed schematic: two LEDs on digital pins 2 and 3.**

**Table 6. Connections**

| From (Arduino) | Through | To |
|---|---|---|
| Digital pin 2 (D2) | R1 = 470 Ω, ¼ W | LED1 anode; LED1 cathode → GND |
| Digital pin 3 (D3) | R2 = 470 Ω, ¼ W | LED2 anode; LED2 cathode → GND |
| GND | - | Common ground rail (LED1 & LED2 cathodes) |

**Not used in the delivered System:** the push-button and 10 kΩ resistor listed in the lab equipment relate only to the Part I button-based Customer concept. The Part II/III System takes its input from the serial-monitor, so no button is wired.

## DESIGN RATIONALE AND ASSUMPTIONS

- Concurrency without an RTOS (EC.5): each LED keeps its own half-period and last-toggle timestamp and is serviced by comparing millis() every loop iteration (the non-blocking “Blink Without Delay” pattern). delay() is never used, so nothing stalls the processor.
- Asynchronous serial (HL.4): the loop reads serial only when characters are available, accumulating them into a line buffer and parsing on a line terminator. Because the loop never waits for input, both LEDs keep blinking at their set rates while a prompt is pending.
- Interval interpretation (HL.5, Table 5): the value entered by The User is the full blink period; on-time = off-time = interval / 2 (e.g. 600 → 300 ms on / 300 ms off), matching Table 5.
- Independence (HL.6): the two LEDs share no timing state, so setting or changing one leaves the other’s rate and phase untouched - reproducing the “LED1/LED2 is unaffected” behavior of Table 5.
- Current limiting: with a ~2 V LED forward drop on a 5 V rail, R = 470 Ω gives (5 − 2)/470 ≈ 6.4 mA per LED - clearly visible and within the ATmega328P per-pin limit (20 mA recommended, 40 mA absolute maximum).
- Assumptions: the serial-monitor sends a newline on Enter; the User enters positive integers; the board runs at 16 MHz; and the LEDs are wired with correct polarity per Figure 1.

### Program flow (pseudocode)

```
setup():
    configure pins 2 and 3 as OUTPUT, drive both LOW
    Serial.begin(9600)

loop():                     // runs continuously, never blocks
    serviceLed(led1)        // toggle LED1 if its half-period elapsed
    serviceLed(led2)        // toggle LED2 if its half-period elapsed
    handleInput()           // print prompt once; read serial non-blocking;
                            //   on a complete line, parse it and update the
                            //   selected LED's half-period
```

The complete source listing is provided in Appendix A.

# TESTING AND VALIDATION

**DESCRIPTION:** The tests below demonstrate to The Customer that The System satisfies the provided high-level requirements (HL.1-HL.6) and educational constraints (EC.1-EC.5). They confirm that two LEDs on pins 2 and 3 blink at independently User-set intervals with on/off equal to half the entered value, that the serial dialog of Table 5 runs correctly, that serial I/O is asynchronous (blinking continues while awaiting input), that changing one LED does not affect the other, and that the software builds with the required toolchain and runs on the Reference-platform without any third-party multitasking library.

## TESTING PLATFORM

1. The testbed of Figure 1: ELEGOO Uno R3 with LED1 on pin 2 and LED2 on pin 3, each through a 470 Ω resistor to a common ground.
2. Arduino IDE 2.3.3+ serial-monitor at 9600 baud, line ending “Newline”.
3. A means of measuring on/off times: a phone camera recording at a known frame rate (e.g. 240 fps slow-motion) or an oscilloscope / logic analyzer on the LED pin (for T.4 and T.5).
4. Host computer with USB and access to the GitHub repository.

## TESTING AND VALIDATION REQUIREMENTS

**Table 7. Testing and Validation Requirements**

| ID | Requirement | Verifies |
|---|---|---|
| **T.0** | All testing and validation must be performed on the testbed of Figure 1 (the Reference-platform), using Arduino IDE 2.3.3 or later. | EC.3 |
| **T.1** | The testbed must contain exactly two LEDs, one on digital pin 2 and one on digital pin 3, each in series with a current-limiting resistor to ground, and the Evaluator must be able to tell LED1 (pin 2) from LED2 (pin 3). | HL.1, EC.3 |
| **T.2** | The System must run on the standard Arduino-compatible ELEGOO Uno R3 with no modification to the board. | HL.2 |
| **T.3** | With the serial-monitor open, the System must prompt for a LED number (1 or 2), then for a blink interval in msec, and must accept both values entered over the serial-monitor. | HL.3, HL.5 |
| **T.4** | For a User-entered interval of N msec, the selected LED must blink with on-time = off-time = N/2 msec, within ±5% measured over 10 cycles (verified with N = 600 → 300/300 and N = 1600 → 800/800 per Table 5). | HL.5 |
| **T.5** | Once set, each LED’s blink interval must remain constant until changed by The User; over ≥ 60 s the period must not drift by more than ±5%. | HL.6 |
| **T.6** | Setting or changing the interval of one LED must not alter the rate or phase continuity of the other LED. | HL.4, HL.6 |
| **T.7** | While the System is awaiting User input at any prompt, both configured LEDs must keep blinking at their set intervals with no pause, stall, or timing disturbance. | HL.4 |
| **T.8** | The System must complete the full example dialog of Table 5 end-to-end when exercised by the Evaluator, producing the LED behavior described there. | EC.4 |

| ID | Requirement | Verifies |
|---|---|---|
| **T.9** | The Evaluator must be able to (a) download all required material from the GitHub repository, (b) cross-compile it with the gcc toolchain of Arduino IDE 2.3.3+ for the ATmega328P, and (c) run the result on the Reference-platform. | EC.1, EC.2, EC.2.1 |
| **T.10** | The delivered software must contain no third-party multitasking library or RTOS; concurrency must be implemented with bare-metal, non-blocking techniques (verified by source inspection and the build log). | EC.5 |

## TESTING RESULTS

Evidence for the tests below is provided by the photograph and videos in this section. Videos are hosted online (GitHub repository or a shared drive) and linked here, since they cannot be embedded in the document.

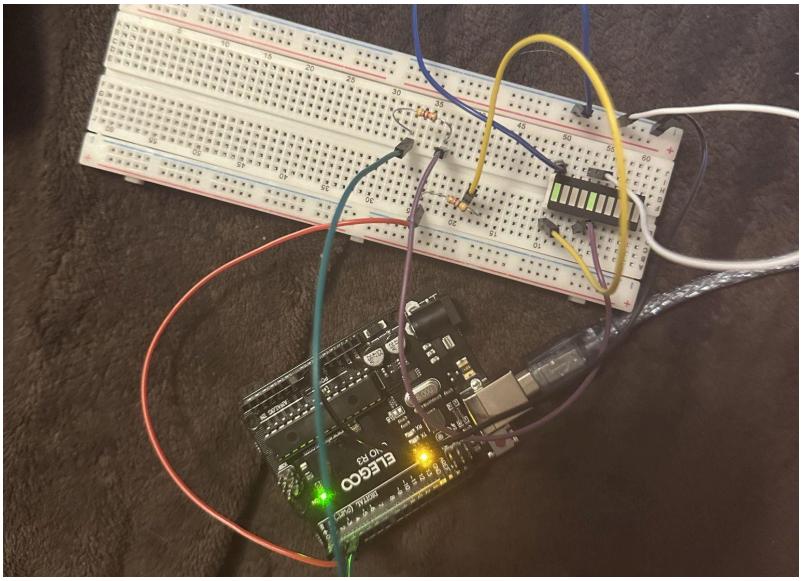


**Figure 2. Photograph of the wired testbed (two LEDs on digital pins 2 and 3).**

**Video 1 -** *Video1* Serial-monitor dialog: entering the Table 5 sequence (LED 2 / 600, then LED 1 / 1600) with the prompts and echoed replies.

**Video 2 -** *Video2* Blink timing with an on-screen timer: 10 full cycles per LED, and the rate held constant over ~60 s.

**Video 3 -** *Video3* Both LEDs blinking independently at their different rates while a new command is typed at the prompt (neither pauses).

**Table 8. Results of tests**

| Test | Result |
|---|---|
| **T.0** | Satisfied: all tests performed on the Figure 1 testbed with Arduino IDE 2.3.3. |
| **T.1** | Satisfied: two LEDs connected to digital pins 2 and 3 through current-limiting resistors (Figure 2). |
| **T.2** | Satisfied: the sketch uploads and runs on the ELEGOO Uno R3 with no modification to the board. |
| **T.3** | Satisfied: the System prompts for the LED number and interval and accepts both over the serial-monitor (Video 1). |
| **T.4** | Satisfied: LED2 at 600 ms measured [__] s over 10 cycles = [__] s/cycle; LED1 at 1600 ms measured [__] s over 10 cycles = [__] s/cycle; on-time = off-time = half the interval, within ±5% (Video 2). |
| **T.5** | Satisfied: each LED's rate stayed constant over ≥ 60 s with no visible drift (Video 2). |
| **T.6** | Satisfied: setting LED1 while LED2 was already blinking left LED2's rate and phase unchanged (Video 3). |
| **T.7** | Satisfied: both LEDs continued blinking at their set intervals while input was typed at the prompt; no pause or stall (Video 3). |
| **T.8** | Satisfied: the full Table 5 dialog was reproduced end to end (Video 1). |
| **T.9** | Satisfied: the source at [insert GitHub repository URL] downloads and compiles with the gcc toolchain of Arduino IDE 2.3.3+ for the ATmega328P and runs on the Reference-platform. |
| **T.10** | Satisfied: the software uses only millis() and Serial from the Arduino core; no RTOS or third-party multitasking library is present. |

## Appendix A. Source code

File: `projectAsync.ino` - also hosted at *https://github.com/dmcomea/CMPE311Fall2026/tree/main/Project1/projectAsync* per EC.2.

```
/*
 * PROJECT-ASYNC : Independent asynchronous control of two LEDs
 * CMPE 311 - UMBC
 *
 * Two LEDs on digital pins 2 and 3 (EC.3) blink independently. The User
 * sets each LED's blink interval (full period, in msec) over the Arduino
 * IDE serial-monitor (HL.3, HL.5). Serial input is handled non-blocking
 * so it never interrupts the blinking (HL.4). No RTOS or 3rd-party
 * multitasking library is used (EC.5): concurrency is achieved with
 * millis()-based non-blocking scheduling.
 */

const uint8_t LED1_PIN = 2;          // EC.3: LED1 on digital pin 2
const uint8_t LED2_PIN = 3;          // EC.3: LED2 on digital pin 3

// Per-LED blink state. halfPeriod = interval/2 (on-time == off-time).
struct Led {
  uint8_t       pin;
  unsigned long halfPeriod;          // msec; 0 = not yet set (stays off)
  unsigned long lastToggle;          // millis() timestamp of last change
  bool          on;
};

Led led1 = { LED1_PIN, 0, 0, false };
Led led2 = { LED2_PIN, 0, 0, false };

// Non-blocking serial input state machine.
enum InputState { ASK_LED, ASK_INTERVAL };
InputState inputState  = ASK_LED;
int        selectedLed = 0;          // 1 or 2, chosen in ASK_LED
char       lineBuf[16];
uint8_t    lineLen  = 0;
bool       prompted = false;         // has the current prompt been printed?

void setup() {
  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);
  Serial.begin(9600);
}

// Advance one LED's blink without blocking (HL.6: rate stays constant).
void serviceLed(Led &led) {
  if (led.halfPeriod == 0) return;            // rate not set -> stay off
  unsigned long now = millis();
  if (now - led.lastToggle >= led.halfPeriod) {
    led.lastToggle = now;
    led.on = !led.on;
    digitalWrite(led.pin, led.on ? HIGH : LOW);
  }
}

// Read available serial chars into lineBuf without blocking.
// Returns true once a full line (terminated by \n or \r) is ready.
bool readLineNonBlocking() {
  while (Serial.available() > 0) {
```

```
    char c = Serial.read();
    if (c == '\n' || c == '\r') {
      if (lineLen == 0) continue;             // ignore stray CR/LF
      lineBuf[lineLen] = '\0';
      lineLen = 0;
      return true;
    }
    if (lineLen < sizeof(lineBuf) - 1) lineBuf[lineLen++] = c;
  }
  return false;                               // nothing complete yet
}

void handleInput() {
  if (!prompted) {                            // print each prompt exactly once
    if (inputState == ASK_LED) Serial.print(F("What LED? (1 or 2) "));
    else                       Serial.print(F("What interval (in msec)? "));
    prompted = true;
  }

  if (!readLineNonBlocking()) return;         // no full line yet -> LEDs keep blinking

  long value = atol(lineBuf);
  Serial.println(value);                      // echo the accepted number

  if (inputState == ASK_LED) {
    if (value == 1 || value == 2) { selectedLed = (int)value; inputState = ASK_INTERVAL; }
    else                          { Serial.println(F("  Please enter 1 or 2.")); }
  } else {                                    // ASK_INTERVAL
    if (value > 0) {
      unsigned long half = (unsigned long)value / 2UL;   // Table 5: 600 -> 300/300
      Led &t = (selectedLed == 1) ? led1 : led2;
      t.halfPeriod = half;
      t.lastToggle = millis();
      t.on = true;
      digitalWrite(t.pin, HIGH);              // begin on-phase immediately
      Serial.print(F("  LED")); Serial.print(selectedLed);
      Serial.print(F(" -> ")); Serial.print(half);
      Serial.print(F(" ms on / ")); Serial.print(half);
      Serial.println(F(" ms off."));
    } else {
      Serial.println(F("  Interval must be a positive number of msec."));
    }
    inputState = ASK_LED;
  }
  prompted = false;                           // trigger the next prompt
}

void loop() {
  serviceLed(led1);       // blinking runs every iteration, independent of input
  serviceLed(led2);
  handleInput();          // non-blocking; never stalls the loop (HL.4)
}
```

**END OF DOCUMENT**